

Отчёт по аудиту безопасности

Был проведён аудит безопасности, посвящённый выявлению и исправлению уязвимостям XSS, Information Disclosure, SQL Injection, CSRF, Include, Upload.

Анализ уязвимостей и защита

XXS: межсайтовый скриптинг

XSS (Cross-Site Scripting) — это тип уязвимости веб-приложений, при котором злоумышленник может внедрить вредоносный JavaScript-код на страницу, которая затем выполняется в браузере других пользователей.

Решение:

1. Экранирование с использованием `<?= htmlspecialchars($data, ENT_QUOTES, 'UTF-8') ?>`

htmlspecialchars — функция в PHP, которая преобразует специальные символы в HTML-сущности для сохранения их значения в HTML-документе, специальные символы (например, `<`, `>`, `&`, `"`) перестают интерпретироваться как HTML-теги.

2. Добавление `<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' cdnjs.cloudflare.com; style-src 'self' 'unsafe-inline'">` в блок `<head>` HTML кода.

Content-Security-Policy — это HTTP-заголовок, который сообщает браузеру, откуда разрешено загружать ресурсы (скрипты, стили, изображения и т.д.), если XSS всё же произойдёт, CSP не даст вредоносному коду выполниться.

Information Disclosure: раскрытие информации

Уязвимость раскрытия информации (Information Disclosure) — это ситуация, когда система или приложение непреднамеренно раскрывает конфиденциальные данные, которые не должны быть доступны неавторизованным пользователям (структуру БД, пути к файлам, исходный код).

Это происходит через прямое логирование ошибок, иногда через комментарии в исходном коде

Решение:

1. Отключение стандартного вывода ошибок

```
error_reporting(0);
ini_set('display_errors', 0);
```

2. Использование общих сообщений об ошибках:

```
catch (PDOException $e) {
    error_log("Database error in admin.php: " . $e->getMessage());
    error_log("Stack trace: " . $e->getTraceAsString());

    die("Произошла ошибка. Администратор уведомлён.");
}
```

SQL Injection: SQL инъекции

SQL-инъекция (SQLi) — это уязвимость в веб-приложениях, которая позволяет злоумышленнику внедрять вредоносный код в запросы к базе данных через пользовательский ввод. Это может привести к несанкционированному доступу к конфиденциальным данным, их изменению, удалению или даже получению полного контроля над системой.

Решение:

Использование подготовленных запросов (уже используется)

```
$stmt = $db->prepare("SELECT * FROM applications WHERE id = ?");
$stmt->execute([$SESSION['user_id']]);
```

При использовании подготовленных запросов сначала формируется шаблон SQL-запроса, а затем отдельно передаются реальные данные на сервер. Сервер автоматически обрабатывает значения, экранируя специальные символы и гарантируя, что они будут рассматриваться как данные, а не как часть исполняемого кода.

CSRF: межсайтовая подделка запроса

CSRF (Cross-Site Request Forgery, межсайтовая подделка запроса) — это вид атак на веб-приложения, при которой злоумышленник вынуждает браузер пользователя выполнить нежелательные действия на доверенном сайте от его имени.

Решение:

Генерация CSRF-токена, проверка CSRF для POST-запросов

```
if (empty($SESSION['csrf_token'])) {
    $SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!isset($_POST['csrf_token']) || $_POST['csrf_token'] !==
$SESSION['csrf_token']) {
        die("Недействительный токен. CSRF атака обнаружена!");
    }
}
```

Мошенник не может узнать токен, его знает только сервер, поэтому может отличать свои запросы от мошеннических.

Проверка токена в форме:

```
<input type="hidden" name="csrf_token" value="<?= $_SESSION['csrf_token'] ?>">
```

Необходим для проверки откуда пришла форма на сервер.

File Include/Upload: включение файлов и загрузка

File Inclusion возникает, когда веб-приложение позволяет пользователю вводить данные, которые используются для динамического включения файлов в код. Это происходит при использовании функций `include()` в PHP.

File Upload возникает, когда веб-приложение принимает файлы от пользователей без должной проверки их типа, содержимого или назначения. Это может позволить злоумышленникам загрузить вредоносные файлы и выполнить их на сервере.

Решение:

Решение для Include - использование статических `include()`:

```
include('admin.php');
```

Классическим решением уязвимости Upload является проверка MIME-типов при загрузке:

```
$allowed_types = ['image/jpeg', 'image/png']; if (!in_array($_FILES['file']['type'], $allowed_types)) die("Недопустимый тип");
```